// ANDROID · PENTEST · FRAMEWORK

alab

Zero-Studio Android Pentest Lab — Install & Operations Guide

// developed by hits · built by human + AI (Opus 4.6)

Device · Pixel 6 · Android 13 · API 33	**Frida** · 17.2.14 · objection · jadx
Root · Magisk 25.2 · Zygisk · DenyList	**Accel** · KVM · HVF · Hyper-V
Proxy · Burp system-cert · 10.0.2.2:8080	**Platforms** · Linux · macOS · Windows
Agents · Claude Code · Gemini CLI ready	**License** · MIT · open source

v2.0 · 2026-05-13

// github.com/hits313/alab

01 Requirements

COMPONENT	MINIMUM	NOTES
OS	Linux / macOS / Windows	All three covered below
CPU	x86_64 VT-x/AMD-V *or* Apple Silicon	Hardware virtualization mandatory
RAM	8 GB system	AVD: 3 GB · Burp + Frida: ~1 GB
Disk	25 GB free	SDK ~6 GB · sysimg ~5 GB · AVD ~10 GB
Python	3.10+	Frida tools · objection · apkleaks
Java	OpenJDK 17	jadx · apktool
Burp	Community / Pro	proxy + CA cert export
Node.js	20+ (optional)	Claude Code · Gemini CLI agents

02 Install — Linux

linux | macos | windows

Tested on Ubuntu 22.04, Pop!_OS, Debian 12. For Fedora/Arch, adjust the package manager.

2.1 · Pre-flight

```
ls /dev/kvm                          # must exist
egrep -c '(vmx|svm)' /proc/cpuinfo   # > 0
free -g                              # ≥ 8
df -h ~                              # ≥ 25 G
```

KVM missing?
`sudo apt install qemu-kvm libvirt-daemon-system && sudo usermod -aG kvm,libvirt $USER` then log out/in.

2.2 · System packages

```
sudo apt update
sudo apt install -y openjdk-17-jdk python3-pip python3-venv \
  qemu-kvm adb scrcpy unzip wget curl git apktool sqlite3 openssl
```

2.3 · Android SDK

```
mkdir -p ~/Android/Sdk/cmdline-tools && cd /tmp
wget https://dl.google.com/android/repository/commandlinetools-linux-11076708_latest.zip
unzip commandlinetools-linux-*.zip
mv cmdline-tools ~/Android/Sdk/cmdline-tools/latest

export ANDROID_SDK_ROOT=$HOME/Android/Sdk
export PATH=$ANDROID_SDK_ROOT/cmdline-tools/latest/bin:$ANDROID_SDK_ROOT/platform-tools:$ANDROID_SDK

yes | sdkmanager --licenses
sdkmanager "platform-tools" "emulator" "platforms;android-33" \
           "system-images;android-33;google_apis;x86_64"
```

Critical: use `google_apis`, **not** `google_apis_playstore`. PlayStore images block `adb root`.

2.4 · Create AVD

```
echo "no" | avdmanager create avd \
  -n Pixel6_API33_root \
  -k "system-images;android-33;google_apis;x86_64" \
  -d "pixel_6"

AVD=~/.android/avd/Pixel6_API33_root.avd/config.ini
sed -i 's/^hw.ramSize=.*/hw.ramSize=3072/' $AVD
sed -i 's/^hw.cpu.ncore=.*/hw.cpu.ncore=6/' $AVD
echo "disk.dataPartition.size=4096M" >> $AVD
```

2.5 · Frida + tooling

```
pip install --user frida-tools==17.2.14 objection apkleaks androguard

mkdir -p ~/tools/android-lab/frida && cd ~/tools/android-lab/frida
wget https://github.com/frida/frida/releases/download/17.2.14/frida-server-17.2.14-android-x86_64.xz
xz -d frida-server-*.xz && mv frida-server-* frida-server && chmod +x frida-server
```

2.6 · jadx · dex2jar · Magisk

```
cd ~/tools/android-lab
wget https://github.com/skylot/jadx/releases/download/v1.5.0/jadx-1.5.0.zip
unzip jadx-1.5.0.zip -d jadx
wget https://github.com/pxb1988/dex2jar/releases/download/v2.4/dex-tools-v2.4.zip
unzip dex-tools-v2.4.zip -d dex2jar

mkdir -p magisk && cd magisk
wget -O Magisk-v28.1.apk https://github.com/topjohnwu/Magisk/releases/download/v28.1/Magisk-v28.1.ap
```

2.7 · Clone alab + aliases

```
git clone https://github.com/hits313/alab.git ~/tools/android-lab/repo
cp ~/tools/android-lab/repo/start-lab.sh ~/tools/android-lab/
chmod +x ~/tools/android-lab/start-lab.sh

# ~/.zshrc
export ANDROID_SDK_ROOT=$HOME/Android/Sdk
export PATH=$ANDROID_SDK_ROOT/platform-tools:$ANDROID_SDK_ROOT/emulator:$HOME/.local/bin:$HOME/tools
alias alab='bash ~/tools/android-lab/start-lab.sh'
```

03 Install — macOS

linux | macos | windows

Tested on macOS 14 Sonoma — Apple Silicon & Intel. HVF replaces KVM and runs out of the box.

3.1 · Homebrew base

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

brew install openjdk@17 python@3.11 wget unzip git \
             android-platform-tools scrcpy sqlite openssl@3
brew install --cask android-commandlinetools

sudo ln -sfn $(brew --prefix)/opt/openjdk@17/libexec/openjdk.jdk \
             /Library/Java/JavaVirtualMachines/openjdk-17.jdk
```

3.2 · Android SDK

```
export ANDROID_SDK_ROOT=$HOME/Library/Android/sdk
mkdir -p $ANDROID_SDK_ROOT
export PATH=$(brew --prefix android-commandlinetools)/bin:$ANDROID_SDK_ROOT/platform-tools:$ANDROID_

yes | sdkmanager --licenses
# Apple Silicon → arm64-v8a · Intel → x86_64
ARCH=$(uname -m); [[ "$ARCH" == "arm64" ]] && IMG="arm64-v8a" || IMG="x86_64"
sdkmanager "platform-tools" "emulator" "platforms;android-33" \
           "system-images;android-33;google_apis;$IMG"
```

3.3 · Create AVD

```
echo "no" | avdmanager create avd \
  -n Pixel6_API33_root \
  -k "system-images;android-33;google_apis;$IMG" \
  -d "pixel_6"
```

3.4 · Frida + tooling

```
pip3 install --user frida-tools==17.2.14 objection apkleaks androguard

mkdir -p ~/tools/android-lab/frida && cd ~/tools/android-lab/frida
ARCH=$(uname -m); [[ "$ARCH" == "arm64" ]] && FA="arm64" || FA="x86_64"
wget https://github.com/frida/frida/releases/download/17.2.14/frida-server-17.2.14-android-$FA.xz
xz -d frida-server-*.xz && mv frida-server-* frida-server && chmod +x frida-server
```

3.5 · Clone alab + zshrc

```
mkdir -p ~/tools && git clone https://github.com/hits313/alab.git ~/tools/android-lab
chmod +x ~/tools/android-lab/start-lab.sh

cat >> ~/.zshrc <<'EOF'
export ANDROID_SDK_ROOT=$HOME/Library/Android/sdk
export PATH=$ANDROID_SDK_ROOT/platform-tools:$ANDROID_SDK_ROOT/emulator:$HOME/.local/bin:$HOME/tools
alias alab='bash ~/tools/android-lab/start-lab.sh'
EOF
source ~/.zshrc
```

macOS gotchas:

- Apple Silicon: **always** use `arm64-v8a` sysimg — x86_64 runs at 5% speed under Rosetta.
- HVF acceleration is automatic; no `/dev/kvm` setup needed.
- First adb session shows "Allow USB debugging" — accept once.

04 Install — Windows

linux | macos | windows

Two paths. WSL2 is **strongly recommended** — it's just the Linux flow with one extra step. Native is supported but bumpier.

4.1 · Option A — WSL2 (recommended)

```
# elevated PowerShell
wsl --install -d Ubuntu-22.04
wsl --update
```

Then inside WSL2, follow the Linux flow (§2). One extra detail:

```
# WSL2 needs explicit KVM passthrough — enable Hyper-V on Windows side
sudo apt install -y qemu-kvm
ls /dev/kvm   # should exist on WSL2 kernel 5.10+
```

To use Burp on Windows with AVD in WSL2:

```
# Windows host IP from inside WSL
ip route show | grep default | awk '{print $3}'   # → 172.x.x.1
# Burp listens 0.0.0.0:8080 on Windows; point AVD proxy at this IP
```

4.2 · Option B — Native Windows

```
# prerequisites via winget
winget install -e --id EclipseAdoptium.Temurin.17.JDK
winget install -e --id Python.Python.3.11
winget install -e --id Git.Git
winget install -e --id 7zip.7zip
```

Download cmdline-tools and unzip to `C:\Android\Sdk\cmdline-tools\latest\`:

```
https://dl.google.com/android/repository/commandlinetools-win-11076708_latest.zip
```

Set env vars (System Properties → Environment Variables):

```
ANDROID_SDK_ROOT  =  C:\Android\Sdk
PATH              +=  C:\Android\Sdk\cmdline-tools\latest\bin
PATH              +=  C:\Android\Sdk\platform-tools
PATH              +=  C:\Android\Sdk\emulator
```

In a **new** PowerShell:

```
sdkmanager --licenses
sdkmanager "platform-tools" "emulator" "platforms;android-33" `
           "system-images;android-33;google_apis;x86_64"

echo no | avdmanager create avd -n Pixel6_API33_root `
  -k "system-images;android-33;google_apis;x86_64" -d "pixel_6"

py -m pip install --user frida-tools==17.2.14 objection apkleaks androguard
git clone https://github.com/hits313/alab.git C:\tools\android-lab
```

`start-lab.sh` is bash — on native Windows, run it via **Git Bash**:

```
# Git Bash
alias alab='bash /c/tools/android-lab/start-lab.sh'
alab start
```

Windows gotchas:

- Hyper-V vs HAXM: Windows 11 → Hyper-V (built-in). Older Windows 10 → install Intel HAXM via SDK manager.
- Same rule: `google_apis`, never `playstore`.
- Git Bash handles `~` and `/c/` — cmd.exe does not.
- Defender may flag `frida-server` + Magisk. Allow-list `C:\tools\android-lab\`.

05 Quick Start

```
alab start          # boot AVD · auto-root · proxy · zygisk
alab setup          # full chain: root + frida + burp-cert + proxy
alab status         # device · root · magisk · frida · proxy
alab stop           # kill emulator
```

Burp wiring · one-time

1. Burp → Proxy → Proxy settings → Import/Export CA → **DER format**
2. Save to `~/tools/android-lab/burp/cacert.der`
3. Run `alab burp-cert` (converts → installs as system cert → reboots)
4. Run `alab proxy-on`

> **Result:** all HTTP/S flows through Burp. No app-level proxy. No user-store cert dance.

06 Command Reference

Boot

COMMAND	ACTION
`alab start`	Boot AVD, auto root, proxy on, zygisk on, denylist on
`alab stop`	Kill emulator
`alab screen`	scrcpy mirror (1080p, screen-off on host)

Root / Magisk

COMMAND	ACTION
`alab root`	`adb root` + remount /system
`alab setup`	Full chain — root + frida + proxy + magisk
`alab magisk`	Daemon version + denylist status
`alab denylist`	Enable Zygisk + DenyList via sqlite3
`alab denylist-add com.x.y`	Add package to DenyList
`alab denylist-ls`	List DenyList entries

Intercept

COMMAND	ACTION
`alab frida`	Push + start frida-server on device
`alab frida-stop`	Kill frida-server
`alab burp-cert`	Install Burp CA as system cert + reboot
`alab proxy-on`	Route all traffic → Burp 10.0.2.2:8080
`alab proxy-off`	Clear proxy

SSL Unpin

COMMAND	ACTION
`alab unpin com.bank.app`	objection spawn + `android sslpinning disable`
`alab unpin-frida com.bank.app`	Frida universal unpin (OkHttp3, TrustKit, TM, WebView, NSC)

APK Analysis

COMMAND	ACTION
`alab install app.apk`	`adb install -r`
`alab pull-apk com.x.y`	Pull installed APK from device
`alab decompile app.apk`	jadx → `/tmp/jadx_<name>/`
`alab strings app.apk`	apkleaks — secrets + endpoints
`alab manifest app.apk`	Dump decoded AndroidManifest.xml

Device

COMMAND	ACTION
`alab screenshot`	Save → `/tmp/screen.png`
`alab logcat com.x.y`	Live logcat filtered by pkg
`alab pull-data com.x.y`	Pull `/data/data/<pkg>`
`alab activities com.x.y`	List activities + exported flag
`alab launch com.pkg/.Main`	`am start -n <component>`
`alab status`	Full status panel

07 Attaching a CLI Agent

The lab is fully scriptable. Both **Claude Code** and **Gemini CLI** can drive `alab` end-to-end — recon, root, unpin, decompile, intercept, report.

7.1 · Claude Code

```
npm i -g @anthropic-ai/claude-code
claude --version
cd ~/tools/android-lab && claude
```

Pre-grant alab perms — drop this into `~/tools/android-lab/.claude/settings.json`:

```
{
  "permissions": {
    "allow": [
      "Bash(alab *)",
      "Bash(adb *)",
      "Bash(frida *)",
      "Bash(frida-ps *)",
      "Bash(objection *)",
      "Bash(jadx *)",
      "Bash(apktool *)",
      "Bash(apkleaks *)"
    ]
  }
}
```

Sample prompts

```
> Boot the lab, install /tmp/target.apk, decompile it, list all
  exported activities, and start frida-server.

> Pull SharedPreferences from com.target.app and look for
  hardcoded API keys or JWTs.

> Hook all java.net.URL constructors in com.target.app via Frida
  and dump them live.

> Run apkleaks on /tmp/target.apk and triage every hit into a
  P1/P2/P3 bucket.
```

7.2 · Gemini CLI

```
npm i -g @google/gemini-cli
gemini auth login
cd ~/tools/android-lab && gemini

> Use the alab framework at start-lab.sh — read it, understand
  the commands, then boot the emulator, install /tmp/target.apk,
  and decompile it with jadx.
```

YOLO / auto-approve (only on trusted alab targets): `gemini --yolo`

7.3 · Tool-call patterns both CLIs follow

PHASE	COMMANDS INVOKED
Bring-up	`alab start && alab setup`
APK staging	`alab install` → `alab decompile`
Static recon	`alab strings` + grep `/tmp/jadx-x/sources`
Manifest review	`alab manifest` + parse exported components
Dynamic hook	`alab unpin-frida com.x.y` · `frida -U -n <p> -l h.js`
Traffic capture	`alab proxy-on` · drive app · read Burp history
Cleanup	`alab proxy-off && alab frida-stop && alab stop`

7.4 · Pro tip — feed the agent the doc

```
claude < ~/tools/android-lab/docs/ALAB-INSTALL.md
# or
gemini -p "$(cat ~/tools/android-lab/docs/ALAB-INSTALL.md) Now boot the lab and decompile /tmp/x.apk
```

08 Troubleshooting

ISSUE	FIX
Emulator slow / black screen	KVM/HVF check · user in `kvm` group · `-gpu swangle` fallback
`adb root` fails	Image must be `google_apis`, not `google_apis_playstore`
Frida version mismatch	`frida --version` == frida-server binary version
Burp cert not trusted	Re-run `alab burp-cert`; system cert needs `-writable-system`
Proxy not intercepting	Burp listener on `0.0.0.0:8080` (AVD uses `10.0.2.2`)
App detects root → exits	`alab denylist-add com.target.app` + objection root-disable hook
`frida-ps -U` empty	Re-run `alab frida` — server dies on reboot
TLS still blocked	`alab unpin-frida <pkg>` — covers OkHttp3, TrustKit, TM, WebView, NSC
Lock file blocks emulator	`rm ~/.android/avd/Pixel6_API33_root.avd/hardware-qemu.ini.lock`
WSL2 `/dev/kvm` missing	Enable Hyper-V on Windows + `sudo apt install qemu-kvm` in WSL
macOS Apple Silicon slow	Use `arm64-v8a` system image, never x86_64

09 Layout

```
~/tools/android-lab/
├── start-lab.sh                # alab dispatcher
├── unpin.sh                    # SSL unpin wrapper
├── magisk-root.sh              # Magisk install helper
├── frida/
│   └── frida-server            # arch-matched binary
├── frida-scripts/
│   └── universal-ssl-unpin.js  # OkHttp3·TrustKit·TM·WebView·NSC
├── jadx/bin/jadx
├── dex2jar/dex2jar/d2j-dex2jar.sh
├── magisk/Magisk-v28.1.apk
├── burp/
│   ├── cacert.der              # exported from Burp
│   └── cacert.pem              # auto-generated
├── apks/                       # drop targets here
└── docs/
    ├── ALAB-INSTALL.md
    └── ALAB-INSTALL.pdf
```